

Лабораторна робота №2

Тема: Графічний інтерфейс користувача

Виконали: ІП-44 Міліщук Сергій, ІП-44 Комін Іван

Мета роботи

Ознайомитися з основами графічного інтерфейсу користувача (GUI) та архітектурою підсистеми GDI (Graphics Device Interface) у середовищі Windows. Вивчити принципи побудови та відображення векторних і растрових графічних примітивів, управління контекстом пристрою, а також розвинути практичні навички реалізації алгоритмів растрової графіки, зокрема рядкового алгоритму заливки довільної області із затравкою.

Завдання до роботи (Варіант 9)

Відповідно до варіанта №9 необхідно:

- Створити віконний Win32-додаток із розміщенням по центру робочого столу.
- Задати замкнений безперервний контур складної конфігурації (довільний неопуклий багатокутник).
- Реалізувати алгоритм заливки області, обмеженої цим контуром, за умови, що точка початку заливки (затравка) визначається довільним кліком лівої кнопки миші всередині фігури.
- Забезпечити коректну перевірку приналежності точки внутрішній області фігури, щоб запобігти виходу заливки за межі контуру.

Хід виконання роботи

1. Ініціалізація вікна та налаштування графічного середовища

- У точці входу wWinMain налаштовано та зареєстровано клас вікна через структуру WNDCLASSEXW. Для забезпечення автоматичної перерисовки вмісту при зміні габаритів вікна встановлено прапорці стилю класу CS_HREDRAW | CS_VREDRAW. Фон вікна призначено стандартну білу кисть (WHITE_BRUSH).
- Функція InitInstance створює вікно розміром 800 на 600 пікселів зі стилем WS_OVERLAPPEDWINDOW. Розрахунок стартових координат виконується за допомогою системних метрик екрана SM_CXSCREEN та SM_CYSCREEN, що забезпечує точне центрування вікна на моніторі.

2. Побудова та відмальовування контуру фігури

- Замкнений контур задано статичним масивом структур POINT (borderPoints), що містить 8 вершин неопуклого багатокутника.
- Усі операції з відтворення контуру виконуються в обробнику події WM_PAINT:

- Виклик `BeginPaint` надає дескриптор контексту пристрою (HDC).
- За допомогою функції `CreatePen` створюється перо товщиною 2 пікселі кольору `BOUNDARY_COLOR` (RGB(128, 128, 128) — сірий колір).
- До контексту пристрою через `SelectObject` обирається створене перо та прозора системна кисть `NULL_BRUSH`. Це дозволяє накреслити виключно межу полігону без попереднього зафарбовування його середини стандартними засобами.
- Багатокутник будується викликом функції `Polygon`, після чого відновлюються попередні графічні об'єкти, а створене перо безпечно видаляється за допомогою `DeleteObject`.

3. Перевірка приналежності точки контуру

Для запобігання запуску алгоритму заливки у відкритому просторі клієнтської зони реалізовано функцію `IsPointInsidePolygon`:

- На основі масиву вершин створюється тимчасовий геометричний регіон викликом `CreatePolygonRgn` з правилом заповнення `WINDING`.
- За допомогою системної функції `PtInRegion` перевіряється, чи лежать координати курсору миші всередині отриманого регіону.
- Дескриптор створеного регіону обов'язково звільняється викликом `DeleteObject` для уникнення витоку дескрипторів GDI.

4. Рядковий алгоритм заливки за кольором межі (Scanline Boundary Fill)

Основний алгоритм реалізовано у функції `BoundaryFill`. Замість класичної рекурсивної моделі (яка спричиняє переповнення програмного стека на великих площах) використано ітеративний рядковий підхід із застосуванням стека `std::stack`:

- **Перевірка точки старту:** Функцією `GetPixel` зчитується колір пікселя в місці натискання. Якщо він збігається з кольором контуру `BOUNDARY_COLOR` або вже зафарбований у `FILL_COLOR` (RGB(0, 150, 255) — яскраво-блакитний), алгоритм одразу завершує роботу.
- **Визначення горизонтального відрізка:**
 - Зі стека вилучається чергова точка-затравка (x, y).
 - Циклами виконується сканування вліво та вправо вздовж поточного рядка у доки не буде досягнуто межі багатокутника, уже зафарбованої області або межі клієнтської зони вікна.
 - У результаті визначаються граничні координати `leftX` та `rightX` неперервного інтервалу, придатного для заливки.
- **Зафарбовування інтервалу:** Усі пікселі знайденого сегмента від `leftX` до `rightX` фарбуються у колір `FILL_COLOR` викликом `SetPixel`.
- **Пошук нових затравок у сусідніх рядках:**
 - Програма перевіряє верхній (y - 1) та нижній (y + 1) рядки в межах діапазону [leftX, rightX].

- За допомогою булевого прапорця `inSegment` фіксуються лише перші пікселі кожного нового незафарбованого підінтервалу, які й додаються в стек як нові затравки. Це радикально мінімізує обсяг збережених у стеку даних.

5. Взаємодія з користувачем

- Подія `WM_LBUTTONDOWN` фіксує клік користувача, координати якого витягуються макросами `LOWORD(lParam)` та `HIWORD(lParam)`.
- Якщо точка лежить у межах фігури, отримується контекст пристрою через `GetDC(hWnd)`, викликається функція заливки `BoundaryFill`, після чого контекст коректно звільняється викликом `ReleaseDC(hWnd, hdc)`.
- При зміні розмірів вікна повідомлення `WM_SIZE` оновлює глобальні змінні габаритів і надсилає запит на повне перемальовування через `InvalidateRect(hWnd, NULL, TRUE)`.

Код програми

Вихідний код проєкту, заголовочні файли та ресурси розміщено у відкритому репозиторії GitHub:

<https://github.com/ivankomin/cpp-labs/tree/main/lab2>

Результати роботи програми

У ході тестування розробленого додатка отримано такі результати:

1. При запуску програми у центрі екрана формується вікно із білим фоном і чітким сірим замкненим контуром складної форми.
2. Клацання лівою кнопкою миші за межами полігону або безпосередньо по сірій лінії контуру блокується й не викликає небажаного зафарбовування зовнішнього простору вікна.
3. Одиначний клік у будь-якій внутрішній області контуру призводить до послідовної повної заливки внутрішнього простору багатокутника блакитним кольором (`RGB(0, 150, 255)`).
4. Алгоритм коректно опрацьовує всі вигини, звуження та западини неопуклого багатокутника без артефактів і «витоку» фарби назовні.
5. Повторний клік по вже зафарбованій внутрішній площині завершується миттєво без виконання зайвих циклів завдяки базовій валідації кольору.

Висновки

Під час виконання лабораторної роботи було опановано засоби графічного інтерфейсу Win32 GDI для роботи з векторними фігурами та растровими даними. Досліджено використання графічних дескрипторів пера (`HPEN`), кисті (`HBRUSH`) та регіонів (`HRGN`), а також закріплено вимоги щодо коректного звільнення системних ресурсів. Успішно реалізовано ефективний стек-орієнтований рядковий алгоритм заливки області за кольором межі (`Scanline Boundary Fill`), який демонструє стабільну роботу без ризику переповнення стека та забезпечує повне заповнення замкнених контурів складної конфігурації.